Gas Tracker

Mobile Application Technology - Final Case Study

1. Project Information

Group members

- Cedric Bryan M. Angulo
- Guia Cyleen C. Torres
- Catherine Joy J. Dela Vega
- Cieldon John P. Obillo

- **Section:** 4C
- **Instructor:** Gel Cunanan
- **Report date:** November 7, 2026
- **Source repository:** gas-tracker-ionic

2. Problem and Proposed Solution

Motorcycle owners can find it difficult to keep consistent fuel records and use them to understand spending and fuel economy. Notes kept in different places do not easily show how one fill-up relates to the next or where a station was visited.

Gas Tracker provides one place for a motorcycle owner to record fill-ups and review personal fuel history. The project adapts ideas from an existing Gas Tracker application used by the developer, which was built with a different technology stack. That reference supports multiple tenants; this case-study version is scoped to personal use, with its account and initial records seeded in the database [3]. The Ionic version adds mobile-focused capabilities, including location-aware map features.

3. Application Features

Main flow

1. Sign in and set the motorcycle's tank and gauge values.
2. Record a fill-up. Amount and date/time are required; liters, odometer, gauge level, station, fuel grade, and location are optional.
3. Review spending and fuel estimates on Home and Charts.
4. Use Log to edit or archive a record. Use Map to review recorded station prices by fuel grade and see where visits are concentrated.

The app supports adding and editing fill-ups, station management, light/dark appearance, and paged lists. Records are archived rather than physically deleted. Map prices come from saved fill-ups; they are not live fuel offers or a current cheapest-station recommendation.

Questions the app helps answer

- How has my fuel spending changed over time?
- What is my recent fuel economy?
- What prices have I recorded at stations I have visited, and when were they recorded?

4. Technical Implementation

The interface uses Ionic 8 and Angular 20. Capacitor 7 provides native access to Geolocation and Haptics. A PHP API validates requests and stores personal fill-up records in MySQL. On the device, Ionic Storage keeps a local snapshot and a queue of unsynchronized changes in IndexedDB. The application uses GET, POST, PUT, and DELETE operations for reading, creating, updating, and archiving records.

The developer built the Android debug APK with Capacitor and Gradle command-line tools, without opening Android Studio. This provided the installable Android app required for the project while avoiding the IDE's resource demands on the developer's laptop. It also lets groupmates test the app by installing the prepared APK instead of configuring a local Android build. The reproducible steps are summarized in the project README.

The installable debug APK is distributed as a downloadable asset on the project's v1.0.0-beta.1 GitHub Release, keeping the binary out of the source repository.

Data flow at a glance

Capture	Save locally	Synchronize	Database
The fill-up form creates a change	IndexedDB keeps the snapshot and retry queue	The app sends queued changes when online	The PHP API validates and writes them to MySQL

Location and feedback

The app requests location only after the user chooses to attach a current location to a fill-up or center the map. It does not track a person in the background. Permission denial, timeout, or unavailable location produces a message; the user can still save without a location. Haptic success feedback is optional and does not interrupt saving if unavailable. These flows are implemented in `src/app/services/geo.ts` [2, 6, 7].

```
if (Capacitor.isNativePlatform()) {
  await Geolocation.requestPermissions({ permissions: ['location'] });
}
const position = await Geolocation.getCurrentPosition({
  enableHighAccuracy: true,
  timeout: 10000,
  maximumAge: 30000,
});
return { latitude: position.coords.latitude, longitude: position.coords.longitude };
```

Local-first data operation

A fill-up is committed to local storage before the app attempts synchronization. This keeps a saved entry available while offline. When a connection returns, queued operations are sent to the API and the app refreshes its account data. Operation IDs make safe retries possible when a response is lost. The server issues short-lived, revocable bearer tokens after sign-in; the account and its records are provisioned in the database. This flow is implemented in `src/app/services/tracker-store.ts` [2].

```
await this.storage.set('snapshot', updated);
this.state.set(updated);
this.channel?.postMessage('updated');
// After the queued local change is committed:
void this.sync();
```

The database separates bike settings, station choices, fuel grades, fill-ups, and mutation receipts. Archived fill-ups retain their history. Local map tiles are not stored, so the map needs an internet connection.

Decision support and data calculations

Home and Charts summarize recorded purchases with price per liter, fuel economy, distance, monthly spending, and an estimated cost per kilometer. Fuel economy uses measured full-tank intervals when available, with a labeled gauge-based estimate as a fallback. Missing measurements are shown as unavailable instead of being treated as real zero values.

The cost-per-kilometer estimate follows the reference app's calculation: use the closing purchase's price per liter and divide by the measured efficiency for that interval. It is an estimate, not the actual cost of every kilometer traveled.

```
segment.to.amount / segment.to.liters / segment.efficiency;
```

This calculation is in `src/app/models/math.ts` [2]. The fill-up form records gauge level after refueling, and the app uses the configured low-fuel anchor and gauge segments when estimating fuel use.

Map comparison

The Map screen uses Leaflet with OpenStreetMap Standard tiles. A heat layer summarizes the frequency of recorded visits, while station cards show saved prices for the selected grade, including an average and the latest recorded price and date. These figures describe the developer's past fill-ups, not current market prices or a reliable prediction of which station is cheapest now. Fuel prices change over time, so comparisons across long periods can be misleading [2, 8, 9].

5. Testing and Evaluation

Automated checks were run on November 7, 2026. Component tests use Angular's experimental test builder with Node and jsdom; map and canvas rendering are isolated. PHP integration checks use a disposable database and do not modify the local development database.

Feature or task	Expected result	Actual result	Status
Fuel calculations and offline queue	Calculations and local changes handle the covered cases	46 logic/offline tests passed	PASS
App components	Components handle the covered form, navigation, chart, settings, log, and map behavior	70 tests across 17 spec files passed in jsdom	PASS
PHP unit and migration behavior	Validation, migration, archival, and restore cases pass	28 tests and 393 assertions passed	PASS
Authenticated API behavior	API validates access and mutations against an isolated database	20 HTTP integration checks passed	PASS
Angular type check	App types compile without errors	`pnpm typecheck` passed	PASS
Main user flow in a browser	Sign in, add a fill-up, review screens, edit, and archive a record	Manually tested by the developer in the localhost web app	PASS
Android install and offline reopening	Install the APK, sign in over HTTPS, and reopen the app while offline	Manually tested by the developer on an installed APK; offline reopening works	PASS
Android location permission	Grant location access and use the device's current position	The manifest includes location permissions; the native permission prompt and location retrieval have not been confirmed in a manual device test	NOT VERIFIED

Follow-up verification on November 7: all 8 focused authentication tests passed, along with Angular type checking and a production build. Capacitor sync and the Android debug APK build succeeded; the packaged manifest includes coarse and fine location permissions. The APK was then manually installed and tested against the HTTPS API, including offline reopening.

The automated checks cover logic, components, and API behavior. The main web flow and installed APK were also manually tested by the developer.

Observed limitations

- The app is scoped to one personal account, which is seeded with initial records in the database.
- Map tiles require internet access.
- The native location permission and current-position flow still needs manual verification on the target Android device.
- Seed data supports manual exploration; some station, location, grade, time, liters, and odometer values are synthetic.
- Station-price summaries combine saved visits from different dates, so seasonal price changes can make long-term averages misleading.

Recommended improvement

Add a selectable time period to station-price comparisons. Comparing prices recorded within a similar period would make the historical summaries more useful while avoiding the impression that an all-time average represents today's cheapest station.

If the app expands beyond personal use, add individual accounts with strict ownership of each rider's records. This would require changes to the database, sign-in flow, API access controls, and account management, but would let multiple riders keep separate fuel histories securely. This was outside the scope of the two-week case study.

6. AI Usage Disclosure

OpenAI Codex assisted with project scaffolding, implementation, debugging, writing and running automated tests, and preparing this report. Railway's agent was used in a limited capacity to speed up backend deployment setup and troubleshooting. The developer provided the main requirements and made the project’s key decisions; the AI tools worked within that direction. Course modules stored in Obsidian provided subject-matter context. The developer manually tested the localhost web app and the installed Android APK connected to the deployed API using HTTPS, including reopening the app offline. The developer reviewed most of the codebase and tested the application. After the developer completes the project and provides the codebase, the group members will review the implementation. Automated unit, component, and API tests were run. No dedicated end-to-end testing library was used.

References

1. Mobile Application Technology - CS, Final Case Study brief supplied for the course.
2. Gas Tracker Mobile source repository: https://github.com/cedricangulo/gas-tracker-ionic. Code areas discussed: `src/app/services/geo.ts`, `src/app/services/tracker-store.ts`, and `src/app/models/math.ts`.
3. Gas Tracker reference application. Private repository maintained in the developer's personal account; access is restricted.

4. Ionic Framework documentation: https://ionicframework.com/docs.
5. Angular v20 component guide: https://v20.angular.dev/guide/components.
6. Capacitor Geolocation API, v7: https://capacitorjs.com/docs/v7/apis/geolocation.
7. Capacitor Haptics API, v7: https://capacitorjs.com/docs/v7/apis/haptics.
8. Leaflet API reference, v1.9.4: https://leafletjs.com/reference-1.9.4.html.
9. OpenStreetMap Foundation, Tile Usage Policy: https://operations.osmfoundation.org/policies/tiles/.
References accessed November 7, 2026.